AI Assisted Coding

Assignment – 7.1

Ch. Nithin Reddy || 2303A51550 || Batch:- 8

Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement)

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., print "Hello"). Use AI to detect and fix the syntax error.

Bug: Missing parentheses in print statement

def greet():

print "Hello, AI Debugging Lab!"

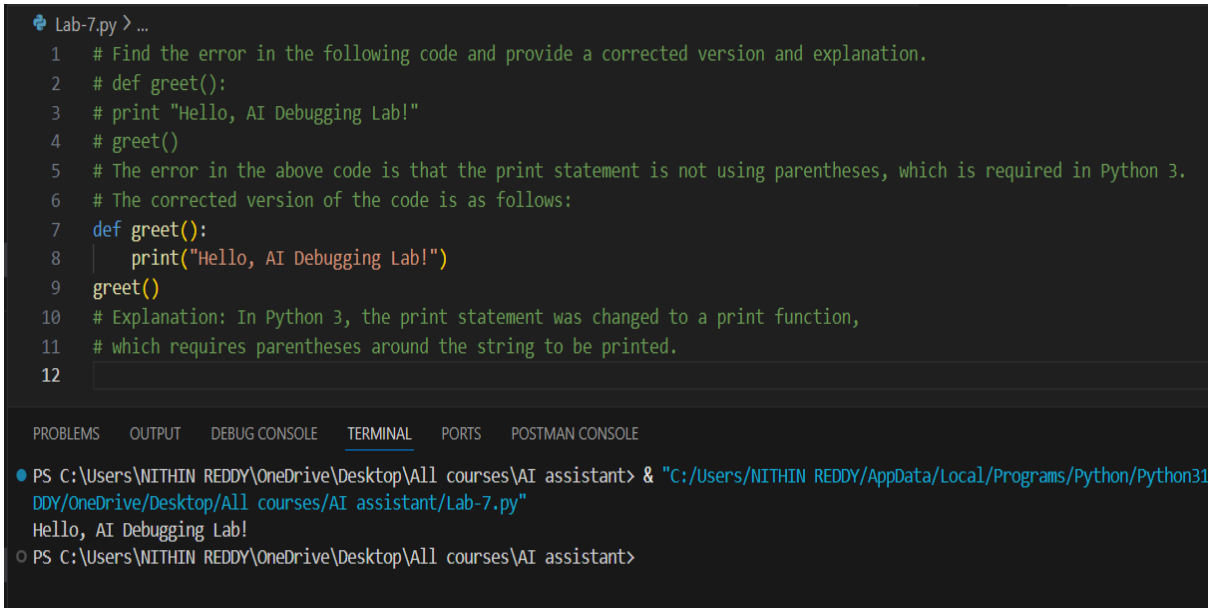
greet()

Requirements:

• Run the given code to observe the error.

• Apply AI suggestions to correct the syntax.

• Use at least 3 assert test cases to confirm the corrected code works.



Task Description #2 (Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses =

instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

def check_number(n):

if n = 10:

return "Ten"

else:

return "Not Ten"

Requirements:

• Ask AI to explain why this causes a bug.

• Correct the code and verify with 3 assert test cases.

```
#fix the error in the if condition for the below provided code and explain the error and the fix.
# def check_number(n):
# if n = 10:
# return "Ten"
# else:
# return "Not Ten"
# The error in the above code is that the assignment operator (=) is used instead of the equality operator (==) in the if condition.
# The corrected version of the code is as follows:
def check_number(n):
    if n == 10:
        return "Ten"
    else:
        return "Not Ten"
# Explanation: In the original code, the if condition is trying to assign the value 10 to n instead of checking if n is equal to 10.
# This will result in a syntax error. In the corrected version, we use the equality operator (==) to check if n is equal to 10,
# which is the intended behavior.
```

Task Description #3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and

crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

def read_file(filename):

with open(filename, 'r') as f:

return f.read()

print(read_file("nonexistent.txt"))

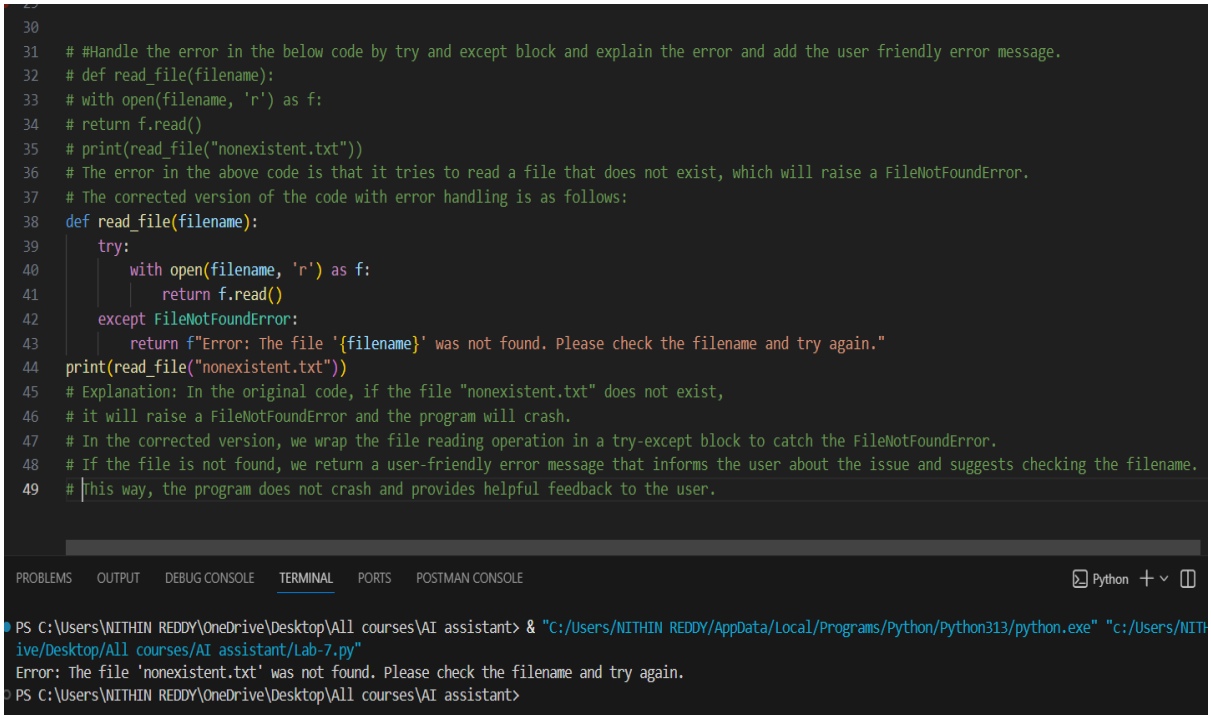
Requirements:

• Implement a try-except block suggested by AI.

• Add a user-friendly error message.

• Test with at least 3 scenarios: file exists, file missing, invalid

path.



Task Description #4 (Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g.,

obj.undefined_method()). Use AI to debug and fix.

Bug: Calling an undefined method

class Car:

def start(self):

return "Car started"

my_car = Car()

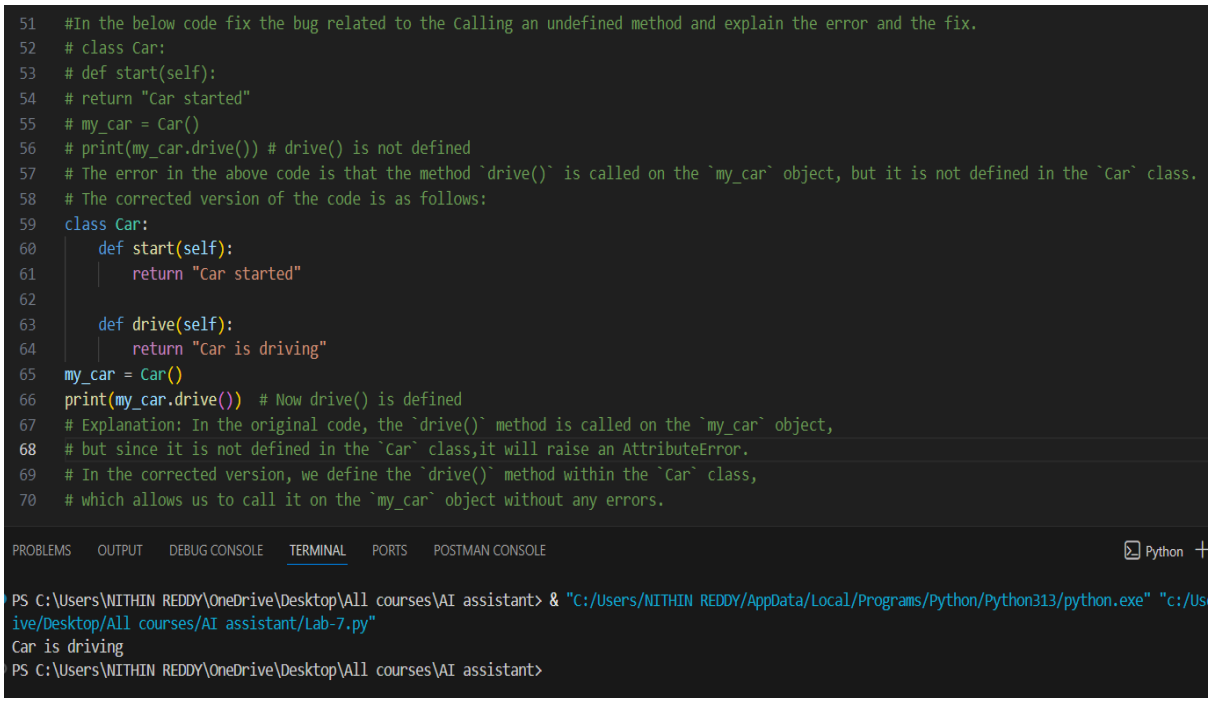
print(my_car.drive()) # drive() is not defined

Requirements:

• Students must analyze whether to define the missing method

or correct the method call.

• Use 3 assert tests to confirm the corrected class works.



```
#In the below code fix the bug related to the Calling an undefined method and explain the error and the fix.
# class Car:
# def start(self):
# return "Car started"
# my_car = Car()
# print(my_car.drive()) # drive() is not defined
# The error in the above code is that the method `drive()` is called on the `my_car` object, but it is not defined in the `Car` class.
# The corrected version of the code is as follows:
class Car:
    def start(self):
        return "Car started"

    def drive(self):
        return "Car is driving"
my_car = Car()
print(my_car.drive())  # Now drive() is defined
# Explanation: In the original code, the `drive()` method is called on the `my_car` object,
# but since it is not defined in the `Car` class,it will raise an AttributeError.
# In the corrected version, we define the `drive()` method within the `Car` class,
# which allows us to call it on the `my_car` object without any errors.
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE                Python +

PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Us
ive/Desktop/All courses/AI assistant/Lab-7.py"
Car is driving
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```

Task Description #5 (TypeError – Mixing Strings and Integers in

Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing

a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

def add_five(value):

return value + 5

print(add_five("10"))

Requirements:

• Ask AI for two solutions: type casting and string

concatenation.

• Validate with 3 assert test cases.

```python
#In the below code there is a TypeError due to mixing string and integer. You fix the error and add that numbers.
# def add_five(value):
# return value + 5
# print(add_five("10"))
# The error in the above code is that it tries to add an integer (5) to a string ("10"), which will raise a TypeError.
# The corrected version of the code is as follows:
def add_five(value):
    try:
        return int(value) + 5
    except ValueError:
        return f"Error: The value '{value}' is not a valid number. Please provide a numeric value."
print(add_five("10"))  # This will now return 15
# Explanation: In the original code, the function `add_five` attempts to add an integer (5) to a string ("10"),
#  which is not allowed in Python and results in a TypeError.
# In the corrected version, we convert the input `value` to an integer using `int(value)`.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python313/py
ive/Desktop/All courses/AI assistant/Lab-7.py"
15
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```